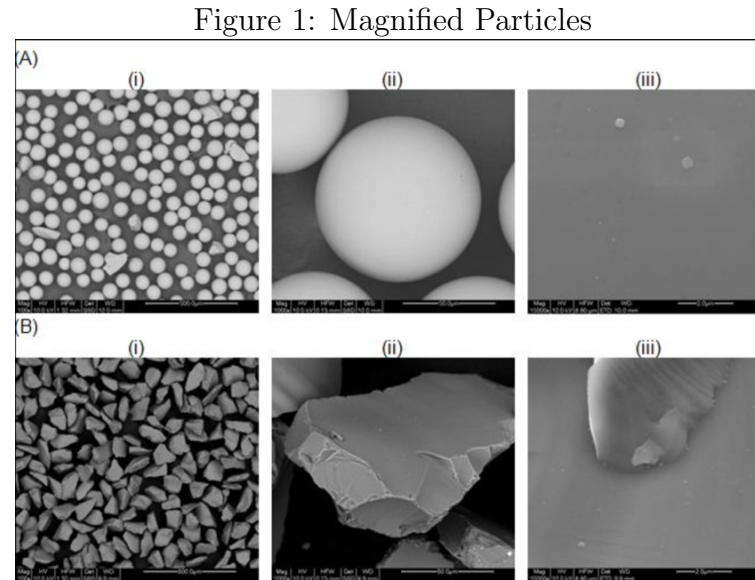


Writing Assignment 5

Avery Woodlief

April 2023

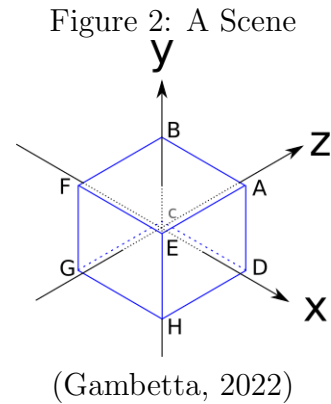
The combination and simultaneous coordination of many minute objects (single textures or simply points in space or in a plane) may be referred to as a Particle System.



(Koh, Micrographs of the Particles at Various Levels of Magnification)

In row A of Figure 1, the three images are what individual particles could look like when zoomed in. However, when looking at the particles at a normal magnification, row B displays images of what the naked eye would see. This dramatic contrast is possible because the particles are so tightly packed together and so small. These systems of particles are designed to simulate events, natural phenomena, or object(s). An advantage of Particle Systems would be the flexibility it has in regards to reproducing the said uses via computer graphics compared to using more complex polygons. For example, in a simulation which uses triangles instead of many points, the appearance of what is being simulated would take more computational power (more strain on the central processing unit) and resources (random access memory, which stores what is currently being run on the computer) to become as apparent since in a triangle there are three points. As a result, using complex polygons for simulations in computer graphics has a significantly greater limitation than that of a Particle System.

Before we can create and use a Particle System, we must first create our world in which the system and all other things will belong in. This world is referred to as a scene and can be portrayed as a cube.



Everything which is created, updated, destroyed, or simply exists is to belong inside of this cube. The faces can be thought of as the world barrier; i.e., nothing is to escape this cube. The reason why we need a scene for our Particle System is to contain things in a finite space which will allow us to manipulate and regulate what happens efficiently without concerning ourselves with unnecessary computation of space outside of what is needed. Once the scene is created, we can begin to make it come alive. This is done by inserting objects into the world and allowing them to perform certain actions in some sort of loop. Figuring out how the Particle System comes into play will be determined with what the system is to be used for. For example, if we are to simulate a body of water such as a river then it is important to determine what happens when objects come into contact with the water. This can be done by using a Particle System.

In Figure 3, at the end of the fishing line, there is a splash. This splash could be simulated via a Particle System to describe the interaction between the object at the end of the fishing line and the water. In this example, the Particle System is not the river, that is simply an animation. The use of the Particle System is to only simulate the splash and is only dependent on two things: the location of where the fishing bob is and if the fishing bob lands in the water.

Notice how the splash in Figure 3 is collectively shaped like a sphere; the particles form a spherical shape when simulating a splash. This spherical shape is known as

Figure 3: Fishing in Skyrim



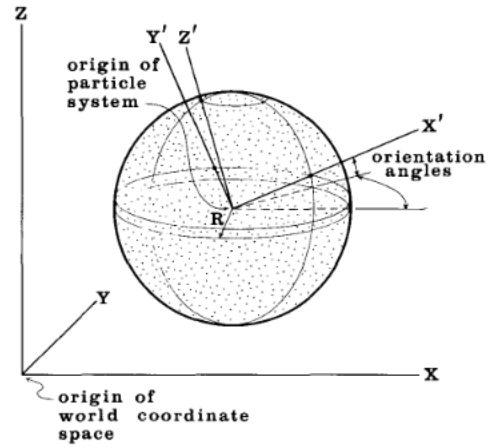
(Hart, 2021)

the generation shape of a Particle System.

For a generation shape of a sphere with radius ρ in space (in relation to the point of initial contact between the river and the fishing bob, which is the origin of the Particle System in Figure 4), a particle is to be placed inside this sphere and this particle would have orientation angles θ and ϕ (Reeves, 1983). For the angle at which it is placed with respect to the positive "x-axis" in the "x-y plane" and the $\pm$ axis of "z", respectively. Where ϕ is to range from $-\pi/2$ to $\pi/2$ and θ is to range from 0 to 2π . In other words, when the fishing bob comes into contact with the river, particles are generated and dispersed in 3-Dimensional space in such a manner as to be able to fit inside of a sphere. Where the furthest particle from the origin of the Particle System is to decide the value of ρ for this generation shape.

In Figure 5, the point $O=(0,0,0)$ and $F=(x_1, y_1, z_1)$, where (x_1, y_1, z_1) is a point inside the sphere. The magnitude of $\mathbf{v}$ is the ρ of our spherical generation shape,

Figure 4: A Spherical Generation Shape Inside Figure 2



(Reeves, 1983)

where $\mathbf{v}=\mathbf{F}$. So, $\mathbf{F}$ is the vector which its magnitude is the radius of the sphere.

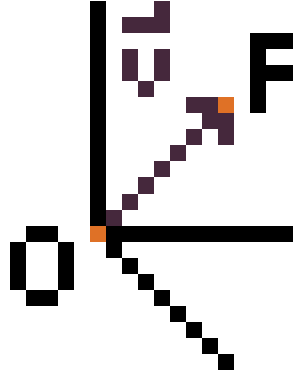
$$\rho = \sqrt{v_1^2 + v_2^2 + v_3^2} = \sqrt{x_1^2 + y_1^2 + z_1^2} \quad (1)$$

In order to calculate the location of an arbitrary particle, we could pick random $a, b, c \in \mathbb{R}$ such that

$$a = rand(0, x_1), b = rand(0, y_1), c = rand(0, z_1) \quad (2)$$

During the duration of the simulation being produced, each particle must have a given lifetime (measured in frames) in order for new particles to be generated while old ones are forgotten or simply to be re-generated instead of creating new particles (which involves more allocation of memory). While the particles are "alive", things will happen to them in order for the particles to present certain behavior. For example, the particles could undergo what are known as stochastic procedures. These

Figure 5: Vector From Origin of Particle System to a Point



(what projection of radius of sphere would look like)

procedures involve certain equations concerning the mean, variance, and a random number in an interval. A general formula to determine a stochastic value for a particle could be presented as:

$$(mean) + (variance * rand()) \quad (3)$$

For example, the number of particles generated in a frame could be equated as:

$$MeanParts_f + Rand() * VarParts_f \quad (4)$$

where $MeanParts_f$ is the mean number of particles for a frame, $Rand()$ is a random number from some interval, and $VarParts_f$ is the variance for the particles of a frame (Reeves, 1983). After the particles in the current frame are updated, they can be rendered into the next frame (the next frame is now the current frame) and the particles before those would be in the frame that is:

$$current_frame_post_update - 1 \quad (5)$$

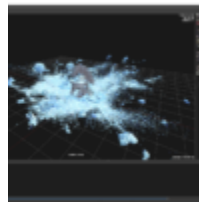
As general as Particle Systems are, there are many applications for which one could be used for. A small list consists of fireworks, fine grain (snow, sand, etc.), representing destruction of objects, explosions, fires, smoke, and rain.

Figure 6: Snow



(Effective TDs Studio Inc.)

Figure 7: Destruction of Arbitrary Object Made of Particles



(Effective TDs Studio Inc.)

Now with the general basics out of the way, it is time to dive into an application of Particle Systems using the information gathered thus far. More specifically, we will be discussing about how to use a Particle System in order to simulate a firework.

First and foremost, in order to simulate a firework, our particles must disperse in a pattern of some sort, ejecting from various angles of θ and ϕ and various values of ρ . Therefore, one approach of this problem would be to simply generate a particle within our sphere (see figure 4) with variable values of angles and length from origin.

$$\rho \in [0, upper_bound_radius] \tag{6}$$

Where the value of ρ in this interval is random.

$$\theta \in [0, 2\pi] \quad (7)$$

Where the value of θ in this interval is random.

$$\phi \in \left[-\frac{\pi}{2}, \frac{\pi}{2}\right] \quad (8)$$

Where the value of ϕ in this interval is random.

Then do this for all the particles in the system. Where the number of particles controls the size of the firework. After the placement of the particles is complete, simply change the color to a single, multiple, or a random color for each particle.

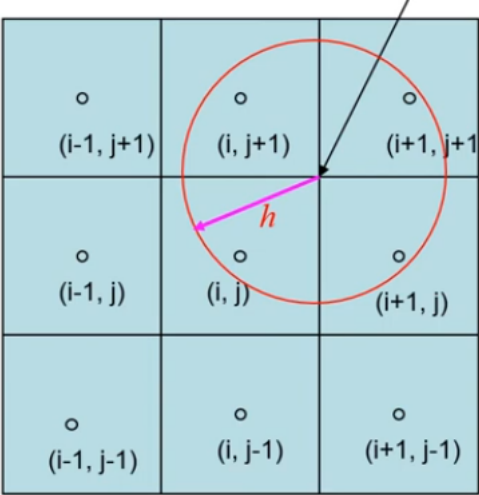
We now turn to a more complicated use of Particle Systems, which is the simulation of water. This kind of simulation requires a technique known as Smooth Particle Hydrodynamics or Smooth Particle Applied Mechanics. We will discuss the derivation of the formula which is used to compute the values of the particles. In order to compute an attribute ****A**** of a particle at a point, we will need to use the neighbors of the particle at that point. Also, we will use a weight which is to be dependent on a distance between two points such that the particle at the point in question is to be equidistant to all the neighbors of that particle. This weight function is expressed as in figure 9 and a simplified version is in figure 8

$$W(r - x, h) \quad (9)$$

where "r - x" is the distance between two particles and "h" is the cut off. Together, the weight can be visualized as
and has the special property

$$\sum_j^k W_j \approx \int_j^k W(r - x, h) dx = 1 \quad (10)$$

Figure 8: Approximating Cell Corner



(Hu, 2021)

where j is first neighbor, k is the last neighboring particle, and dx is differential of the position.

As a result of this special property, we can use the dirac delta function,

$$\delta(x)$$

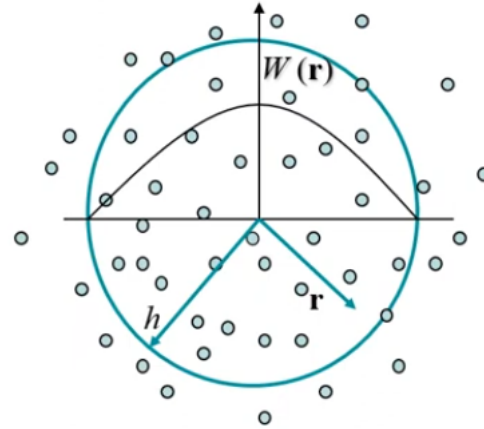
, instead to further approximate the attribute of the particle in question. This is because

$$\int_a^b \delta(x) \, dx = 1 = \int_j^k W(r - x) \, dx$$

The

$$\delta(x)$$

Figure 9: The Weight



(Hu, 2021)

has the special property

$$\int_a^b A(x) \delta(r - x) dx = A(r)$$

as such, we can use the formula

$$\int_j^k A(x) * W(r - x, h) dx = A(r) \tag{11}$$

so, the attriute A of a particle at position i can be approximated as

$$A(r) \approx A_i = \sum_j^k A_j * W_{ij} * dV \tag{12}$$

$$dV = \frac{m_j}{p_j} \tag{13}$$

where m is mass and p is density of a particle at point j.

$$A_i = \sum_j^k \frac{A_j * W_{ij} * m_j}{p_j} \tag{14}$$

Bibliography

1. Gomes, Abel. “Tecnologias De Jogos De Vídeo.” Departamento De Informática, <http://www.di.ubi.pt/agomes/tjv/teoricas/09-particles.pdf>.
2. Reeves, William T. Particle Systems a Technique for Modeling a Class of Fuzzy Objects - LRI. July 1983, <https://www.lri.fr/perso/mbl/ENS/IG2/devoir2/files/docs/fuzzyParticles.pdf>.
3. Toronto, Alain Fournier Univ. of, et al. “Computer Rendering of Stochastic Models.” Communications of the ACM, 1 June 1982, <https://dl.acm.org/doi/10.1145/358523.358553>.
4. Abraham, Ralph H., and Christopher D. Shaw. Home Page - Portal IFSC. 1992, https://www.ifsc.usp.br/reynaldo/curso_caos/Dynamics.pdf
5. “2D Particle System.” [”2D Particle System”], <https://nintervik.github.io/2D-Particle-System/>.
6. Nintervik. “Nintervik/2D-Particle-System: A 2D Particle System Done with C++ and SDL 2.0.” GitHub, <https://github.com/nintervik/2D-Particle-System>.
7. Eck, David J. “Introduction to Computer Graphics.” Introduction to Computer Graphics – Title Page, 20 Dec. 2021, <https://math.hws.edu/graphicsbook/>.
8. Gambetta, Gabriel. “Computer Graphics from Scratch.” Computer Graphics from Scratch - Gabriel Gambetta, 2022, <https://gabrielgambetta.com/computer-graphics-from-scratch/>.
9. Koh, P.T.L. “Micrographs of the Particles at Various Levels of Magnification. (a ...” Micrographs-of-the-Particles-at-Various-Levels-of-Magnification-A-Spherical-particles_fig3_268060903, https://www.researchgate.net/figure/Micrographs-of-the-particles-at-various-levels-of-magnification-A-Spherical-particles_fig3_268060903.
10. Hart, Aimee. “Skyrim Fishing: How to Get a Fishing Rod, Fishing Spot Locations and Fish List in Skyrim: Anniversary Edition.” Eurogamer.net, Eurogamer.net, 25 Nov. 2021, <https://www.eurogamer.net/skyrim-fishing-how-to-unlock-rod-fish-locations-8044>.

11. “Storm 0.5.4 // the Simulation Tool.” Effective TDs Studio Inc., <https://effectivetds.com/resources/fx-tools/storm/>.
12. Max, N. “A Particle-Based Model for Water Simulation.” A Particle-Based Model for Water Simulation (Conference) — OSTI.GOV, 1 Jan. 1998, www.osti.gov/servlets/purl/310908.
13. Hu, Xiangyu. “Introduction on SPH Method.” YouTube, 8 Feb. 2021, www.youtube.com/watch?v=2RoITtjpE-w.